Design Pattern Analysis in SvelteKit-Based Smart Home (ha-fusion)

Part 3: Cross-Language Implementation & Empirical Benchmark Comparison (Team A4)

Java (OpenJDK 23) • Python 3.11 • JavaScript (Node.js 22 ESM) • C++ (MinGW GCC 6.3 -O2 -std=c++14)

Pattern	Collaborative Context	Java			Python			JavaScript			C++		
		LOC	CC	ms	LOC	CC	ms	LOC	CC	ms	LOC	CC	ms
Composite	Visibility Condition Tree	89	8	6.912	64	8	0.024	76	3	0.310	53	8	0.047
Observer	WebSocket Entity Subscriptions	46	3	4.576	41	2	0.024	45	3	0.567	40	3	0.029
Memento	Hearth History & Edit Transaction	87	5	2.520	64	4	0.039	70	6	0.170	11	3	0.027
Strategy	Condition Evaluator Dispatch	60	7	22.374	86	8	0.054	56	3	0.395	57	8	0.055
Factory Method	Entity Domain Modal Factory	63	5	8.894	55	5	0.018	71	5	0.141	38	5	0.032
Template Method	Konva Canvas Renderer Base	63	2	3.899	48	2	0.009	50	2	0.221	46	1	0.036
Total / Mean	*System-Wide Averages*	**408**	**5.0**	**8.196**	**358**	**4.8**	**0.028**	**368**	**3.7**	**0.301**	**245**	**4.7**	**0.038**

Java 23 (OOP / JVM)

- **Type Safety:** Strict interfaces & generics.
- **Structure:** Explicit classes, zero duck typing.
- **Footprint:** Highest LOC (408, +66% vs C++).
- **Latency:** 2.52–22.37 ms (JVM JIT warmup).
- **Fidelity:** 100% test pass (6/6, 5/5).

Python 3.11 (Dynamic / ABC)

- **Idiom:** ABC interfaces & native dicts.
- **Velocity:** First-class lambdas & closures.
- **Footprint:** 358 LOC (-12% vs Java).
- **Latency:** 0.009–0.054 ms (fast micro-exec).
- **Fidelity:** 100% test pass (6/6, 5/5).

JavaScript (Node 22 ESM)

- **Origin:** Direct ha-fusion architectural peer.
- **Idiom:** Prototypes, ES modules & Map store.
- **Footprint:** 368 LOC (-10% vs Java).
- **Latency:** 0.141–0.567 ms (V8 IC runtime).
- **Fidelity:** 100% test pass (6/6, 5/5).

C++14 (Native / RAII)

- **Zero Overhead:** std::shared_ptr & vtables.
- **Control:** Deterministic memory & templates.
- **Footprint:** Lowest LOC (245, -40% vs Java).
- **Latency:** 0.027–0.055 ms (Windows QPC).
- **Fidelity:** 100% test pass (6/6, 5/5).

Key Architectural Findings & Engineering Tradeoffs:

1. **Complexity Invariance (Lizard CCN):** Cyclomatic complexity remains consistent across languages for identical business algorithms (e.g., Composite condition branching = 8 in Java, Python, and C++). Syntax variations alter boilerplate LOC, not intrinsic decision complexity.
2. **Performance Hierarchy:** C++ (0.027–0.055 ms) and Python (0.009–0.054 ms) demonstrate optimal micro-benchmark execution, followed by Node.js V8 (0.141–0.567 ms). Java exhibits startup/JIT warm-up latency (2.52–22.37 ms) on micro-tests.
3. **Domain Matching:** JavaScript/TypeScript is the natural fit for ha-fusion UI rendering; Python provides unmatched developer velocity for automation engines; C++ provides bounded memory execution for IoT edge gateways.